# Azure Cosmos DB — Conceptual and Practical Overview

1. What is Azure Cosmos DB?

**Azure Cosmos DB** is a **fully managed, globally distributed, multi-model NoSQL database service** provided by Microsoft. It is designed for applications that require

- Very low latency
- Elastic scalability
- High availability
- Global distribution
- Guaranteed performance

It is commonly used in **cloud-native**, **microservices**, **IoT**, **real-time analytics**, and **high-scale web/mobile applications**.

---

**2. Key Problems Cosmos DB Solves**

Traditional relational databases often struggle with:

- Horizontal scaling
- Global replication
- Low-latency access across regions
- Handling massive unstructured or semi-structured data

Azure Cosmos DB addresses these challenges by offering:

- Automatic scaling
- Multi-region writes
- Schema-less data models
- Built-in replication and failover

---

### 3. Core Architecture Concepts

#### a. Account

- Top-level Azure resource
- Defines regions, consistency, and APIs

#### b. Database

- Logical container for data
- Can contain one or more containers

#### c. Container (Critical Concept)

- Stores items (documents/rows)
- Automatically partitioned
- Provisioned with throughput (RU/s)

#### d. Item

- Individual record (JSON document in most APIs)
- Must have a unique id

---

### 4. Data Models and APIs

Cosmos DB is **multi-model**, meaning it supports different data models through APIs:

| API | Data Model | Common Use Case |
|---|---|---|
| Core (SQL) API | JSON documents | Most common (Web APIs, Microservices) |
| MongoDB API | BSON documents | MongoDB-compatible apps |
| Table API | Key-Value | Simple structured data |
| Cassandra API | Column-family | High-write workloads |
| Gremlin API | Graph | Relationship-heavy data |

- In ASP.NET Core and Azure Functions, the **Core (SQL) API** is most frequently used.

---

**5. Global Distribution & Replication**

Cosmos DB allows you to:

- Replicate data to **any Azure region**
- Enable **single-write** or **multi-write** regions
- Automatically handle **failover**

Benefits:

- Low latency for global users
- High availability (99.999% SLA with multi-region writes)
- Disaster recovery without custom logic

---

**6. Consistency Models (Very Important)**

Cosmos DB provides **five well-defined consistency levels**, allowing you to balance performance and data accuracy:

| Consistency | Guarantee |
|---|---|
| Strong | Linearizability (like SQL Server) |
| Bounded Staleness | Lag with time/version limits |
| Session | Consistent per user session (default) |
| Consistent Prefix | Order preserved |
| Eventual | Fastest, least strict |

- Most enterprise APIs use **Session Consistency** for optimal balance.

---

**7. Throughput and RU/s (Request Units)**

In **Azure Cosmos DB**, **throughput** defines **how many operations your database or container can handle per second**.

Cosmos DB measures throughput using **RU/s (Request Units per second)**.

Cosmos DB uses **RU/s (Request Units per second)** as a performance currency.

- Every operation (read/write/query) consumes RU
- Cost depends on:
    - Item size
    - Query complexity
    - Index usage

Example:

- Simple point read: ~1 RU
- Complex query: 10–100+ RU

Provisioning options:

- Manual RU/s
- Autoscale RU/s

---

## 8. Partitioning (Must-Understand Topic)

**Why Partitioning?**

To scale horizontally and distribute load.

Partitioning enables **horizontal scaling**

**Partition Key**

- A property in your item (e.g., Category, UserId)
- Determines how data is distributed

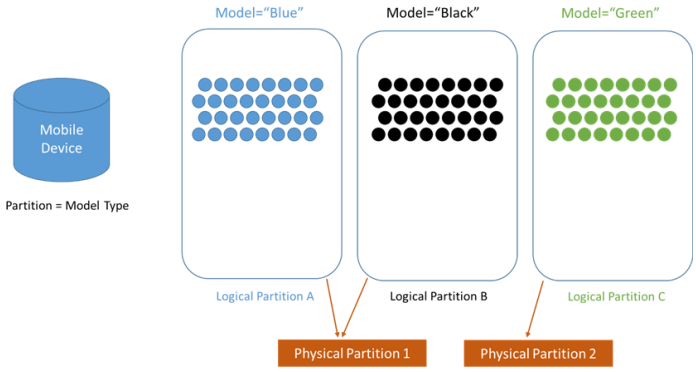
Good partition key characteristics:

- High cardinality
- Even data distribution
- Used frequently in queries

Bad partition key example:

- Constant values (e.g., Country = India)
- Leads to **hot partitions**

---




### 9. Indexing

Cosmos DB automatically indexes all properties by default.

Advantages:

- No index management
- Fast queries out-of-the-box

Customization:

- Include/exclude paths
- Reduce RU cost for writes
- Optimize query performance

---

### 10. Security Features

Cosmos DB supports enterprise-grade security:

- Azure AD authentication
- Role-Based Access Control (RBAC)
- Keys and connection strings
- Private endpoints and VNET integration

- Encryption at rest and in transit

---

**11. Programming Model (ASP.NET Core Perspective)**

In .NET applications:

- Use Microsoft.Azure.Cosmos SDK
- Operations are **async**
- Container-based access

Common operations:

- Create item
- Read item by id + partition key
- Query using SQL-like syntax
- Update / Delete item

This aligns well with:

- Repository pattern
- Clean Architecture
- CQRS (which you are already working with)

**12. Cosmos DB vs SQL Server (High-Level)**

| **Feature** | **Cosmos DB** | **SQL Server** |
|---|---|---|
| Schema | Schema-less | Schema-based |
| Scaling | Horizontal | Vertical (mostly) |
| Global distribution | Built-in | Complex |
| Latency | Single-digit ms | Higher |
| Transactions | Limited scope | Full ACID |
| Cost model | RU-based | Resource-based |

---

**13. Common Use Cases**

- Microservices data store
- Product catalogs
- User profile storage
- Event sourcing
- IoT telemetry
- Gaming leaderboards

---

**14. When NOT to Use Cosmos DB**

Avoid Cosmos DB if:

- Strong relational joins are required
- Complex transactions across tables are mandatory
- Workload is small and static
- Cost sensitivity is high without scale needs

---

**15. Mental Model to Remember**

**Cosmos DB is not a replacement for SQL Server.**
It is a **globally distributed, highly scalable NoSQL database** optimized for **cloud-scale applications**.